

User Manual – Safe Semantic Planner

PCCST503 – Machine Learning, Assignment 1

1. Requirements

C++17 compiler such as g++/GCC, Clang, or MSVC. No third-party libraries are required.

2. Build

Linux/macOS:

```
g++ -std=c++17 -O2 -Wall -Wextra safe_planner.cpp -o safe_planner
```

Windows with MinGW:

```
g++ -std=c++17 -O2 -Wall -Wextra safe_planner.cpp -o safe_planner.exe
```

3. Run

```
./safe_planner
```

The program runs Tests 1–6 and prints success, state path, transition path, raw path cost, minimum bad-state distance, number of explored states, and planning time.

4. Using the Planner Class

Create a `PlanningProblem`, fill `states/transitions/badStates`, then call `SafeLPAStar::plan(problem)`. The class also exposes `setTransitionAvailable()`, `setGoal()`, `setBadStates()`, `addTransition()`, and `currentPlan()` for incremental replanning.

5. Important Parameters

`SafeLPAStar(lambdaSafety, lambdaReliability)`. Set both to zero for pure transition-cost planning. Increase `lambdaSafety` to place greater emphasis on distance from bad states and transition safety; increase `lambdaReliability` to penalize unreliable transitions.

6. Output Interpretation

`success=true` means a valid route to the current goal was found. The minimum bad-state distance is the minimum Euclidean distance from any visited state to the nearest bad state.

7. Source File

`safe_planner.cpp` contains the complete implementation and test driver.